Software Engineering
Answers — One by One

Based on the syllabus page shared by Sajeeb Khan on 3 November 2026. Each topic is answered in the same order as the syllabus, in clear, exam-ready English. Use these answers to understand the idea first, then rewrite the key points in your own words for an exam.

Prepared for: Sajeeb Khan | Topics covered: 24 | Parts: 4

Contents

- Part 1 — System Modeling & Design Engineering (Questions 1–10)
- Part 2 — Software Testing & Evolution (Questions 11–18)
- Part 3 — Dependability & Security (Questions 19–21)
- Part 4 — Software Management (Questions 22–24)

Part 1 — System Modeling & Design Engineering

1. Structural Models

A structural model shows the static structure of a system: what the parts are, and how they are connected. It does not show the order of actions over time; that is a behavioural model.

- In object-oriented design, the main structural model is the class diagram: classes, their attributes and operations, and relationships such as association, aggregation, composition, inheritance, and dependency.
- Other structural views include component diagrams and deployment diagrams, which show software components and the hardware they run on.
- Example: in a library system, Book, Member, and Loan are classes. A Member borrows many Books through a Loan. The diagram makes this structure clear before any code is written.
- Why it matters: a good structural model finds missing parts, wrong relationships, and overly complex classes early, when they are cheap to fix.

2. Design and Implementation, Object-Oriented Design (OOD)

Design is deciding how the system will be built; implementation is writing the code that follows that design. Object-oriented design (OOD) does this by modelling the system as interacting objects, each with data (state) and behaviour (methods).

- OOD starts from the requirements and an object model, then refines it into classes, interfaces, and collaborations that can be coded directly.
- Design and implementation are not strictly separate: coding often reveals design problems, so the design is updated as implementation proceeds. This is normal in iterative development.
- Good OOD aims for high cohesion inside a class (its parts belong together) and low coupling between classes (a change in one class affects few others).
- Example: a BankAccount class keeps balance private and offers deposit() and withdraw() methods, instead of letting every other part of the program change the balance directly.

3. Architectural Design, Views and Decisions

Architectural design is the first, high-level design step. It decides the overall structure of the system: its main components, how they communicate, and the major technical choices. Later design fills in the details inside each component.

- Common architectural styles include layered architecture, client-server, microservices, pipe-and-filter, and repository (shared database) architecture. The choice depends on performance, security, scalability, and modifiability needs.
- Architectural views show the same system from different angles: the logical view (classes and components), the process view (concurrency), the development view (code organisation), and the physical view (deployment on hardware). No single view shows everything.
- Architectural decisions are hard to change later, so they must be recorded with their reasons and trade-offs. Example: choosing microservices improves independent scaling, but adds complexity in communication, testing, and deployment.
- A bad early architecture causes problems in every later stage, while a good one makes the system easier to understand, test, and change.

4. Software Design Approaches

A software design approach is a general strategy for breaking a large problem into smaller, manageable parts and then building a solution from them.

- The two classic approaches are top-down design and bottom-up design (covered in Questions 5 and 6). Most real projects use a mix of both.
- Other approaches include object-oriented design, function-oriented design, and component-based design, where existing reusable components are combined.
- The approach should match the problem: a well-understood problem suits top-down decomposition; a problem with many reusable low-level parts suits bottom-up or component-based work.
- Whatever the approach, the result should be modular: each part has a clear purpose, a clear interface, and can be understood and tested on its own.

5. Top-Down Design Approach

Top-down design starts from the whole system and breaks it down step by step into smaller subsystems, then modules, then functions, until each piece is simple enough to implement.

- Step 1: state the main problem. Step 2: split it into major functions. Step 3: split each function again, until the lowest level is a single, clear task.
- Advantages: the overall structure is clear from the start, the design matches the requirements, and different teams can work on different modules in parallel.
- Disadvantages: low-level difficulties may only appear late, and early assumptions about the top level may be wrong.
- Example: a university system is split into Admission, Examination, and Library modules. Examination is then split into Result Entry, Grade Calculation, and Report Generation.

6. Bottom-Up Design Approach

Bottom-up design starts from the smallest, basic building blocks and combines them into larger subsystems, and finally into the complete system.

- Step 1: identify low-level components that are already available or easy to build, such as data structures, utility functions, or library components. Step 2: combine them into subsystems. Step 3: combine subsystems into the full system.
- Advantages: reusable components are tested early, low-level problems are found early, and the design is grounded in what actually works.
- Disadvantages: the overall structure can become unclear, and the final system may not match the original requirements well, because it grew from the parts upward.
- Example: first build tested functions for sorting, searching, and file handling, then use them to build a student record module, then use that module in a full university system.
- In practice, top-down and bottom-up are combined: the top level gives direction, while reusable bottom-level components save time and reduce risk.

7. Cohesion and High Coupling

Cohesion and coupling measure the quality of a modular design. Good design aims for high cohesion and low coupling. High coupling is a problem, not a goal.

• Cohesion is how closely the parts inside one module belong together. High cohesion means a module does one well-defined job. Low cohesion means it does several unrelated jobs and is hard to understand, test, and reuse.

• Coupling is how much modules depend on each other. Low coupling means a module can be changed with little effect on others. High coupling means a change in one module forces changes in many others, which increases cost and the risk of new bugs.

• Types of cohesion, from best to worst, include functional cohesion (one task), sequential cohesion, communicational cohesion, procedural cohesion, temporal cohesion, logical cohesion, and coincidental cohesion (unrelated parts grouped together).

• To improve a design: split a low-cohesion module into smaller focused modules, hide internal details behind clear interfaces, and reduce shared data between modules.

8. Best Software Design Reduces Long-Term Development Cost

Good design costs more time at the start, but it greatly reduces the total cost over the life of the software, because most cost comes after the first release, in maintenance and change.

• A clear, modular design makes bugs easier to find and fix, because a fault is usually contained in one module with a clear interface.

• It makes changes cheaper: a new feature or a changed requirement can be added by changing one module, not by rewriting the whole system.

• It improves reuse: well-designed components can be used in other projects instead of being built again.

• It reduces communication cost in a team, because each developer can work on a module by understanding its interface, not the whole system.

• By contrast, a rushed, tangled design may deliver quickly at first, but every later fix becomes slower and riskier. This is often called technical debt: short-term speed borrowed at long-term cost.

9. Five Activities of Object-Oriented Design

Object-oriented design is not a single step. It is usually described as five related activities, repeated as understanding improves.

• 1. Understand and define the context: identify the system boundary, its users, and the other systems it interacts with.

• 2. Design the system architecture: choose the overall structure, major components, and how they communicate.

• 3. Identify the principal objects: find the key classes from the requirements, such as real-world entities, and define their responsibilities.

• 4. Develop design models: refine the classes with attributes, operations, relationships, and collaborations, often using UML diagrams.

• 5. Specify interfaces: define exactly how objects and components interact, including method signatures, messages, and data formats, so they can be implemented and tested independently.

• These activities are iterative: a problem found in interface design may require going back and changing the architecture or the object model.

10. Encapsulation, Inheritance, and Polymorphism

These are the three core ideas of object-oriented programming. Together they make code more organised, reusable, and safe to change.

- Encapsulation: bundling data and the methods that work on that data inside one class, and hiding the internal details from other parts of the program. Other code must use public methods, so internal changes do not break the whole system. Example: a BankAccount hides its balance and only allows deposit() and withdraw().
- Inheritance: a new class can reuse and extend an existing class. The child class inherits attributes and methods from the parent class and can add or override behaviour. Example: Student and Teacher both inherit from Person, but add their own fields.
- Polymorphism: the same interface can behave differently for different objects. Example: a draw() method works for Circle, Square, and Triangle, but each draws itself differently. The calling code does not need to know which shape it is.
- Together: encapsulation protects data, inheritance avoids duplication, and polymorphism makes code flexible and extensible.

Part 2 — Software Testing & Evolution

11. Release and User Testing

Release testing and user testing are the final testing stages before, and sometimes after, a system is given to real users. They check the complete system in a realistic environment, not just individual parts.

- Release testing (also called acceptance testing in many contexts) checks that the system meets its requirements and is ready for release. It is usually done by a test team that is independent of the developers.
- User testing involves real users working with the system on real tasks. Alpha testing is done by users at the developer's site; beta testing is done by users at their own sites, with a pre-release version.
- The goal is different from developer testing: to find problems in usability, performance, configuration, and fit to real work, which developers may miss because they know the system too well.
- A system can pass all unit tests and still fail user testing if it is confusing, slow, or does not match how users actually work. That is why user testing is essential before full release.

12. Cyclomatic Complexity

Cyclomatic complexity is a measure of how complex a program's control flow is. It counts the number of independent paths through a piece of code, and it is used to judge how hard the code is to understand and test.

- For a control-flow graph, cyclomatic complexity V(G) = E - N + 2P, where E is the number of edges, N is the number of nodes, and P is the number of connected components (usually 1 for a single program).
- A simpler way for structured code is: V(G) = number of decision points (if, while, for, case, and) + 1.
- Example: a function with no decisions has complexity 1. A function with one if statement has complexity 2, because there are two paths: the condition true, and the condition false.
- Why it matters: higher complexity means more test cases are needed for full path coverage, the code is harder to understand and maintain, and the risk of bugs is higher. A common guideline is to keep a function below 10, and to split it if it grows much larger.

13. Software Evolution, Maintenance and Legacy Systems

Software evolution is the process of changing software after its first release, to fix faults, adapt to new environments, and add new features. Maintenance is the work of keeping software useful over its lifetime, and it is usually the largest part of total cost.

- Types of maintenance: corrective maintenance (fixing faults), adaptive maintenance (adapting to a new operating system, hardware, or law), perfective maintenance (improving performance or adding features at the user's request), and preventive maintenance (changing code to prevent future problems).
- Lehman's laws of evolution observe that a useful system must keep changing or it becomes less useful, and that each change makes the system more complex unless work is done to control that complexity.

• A legacy system is an old system that is still essential to a business, but is hard and expensive to maintain, often because it uses old technology, has poor documentation, and its original developers have left.

• Options for a legacy system include: keep maintaining it, re-engineer it (keep the function, rebuild the structure), replace it with a new system, or wrap it so new systems can use it through an interface. The choice depends on its business value and its technical quality.

14. Functional Testing and Performance Testing

Functional testing and performance testing check different qualities of a system, and both are needed before release.

• Functional testing checks what the system does: does each function produce the correct output for given inputs, according to the requirements? Techniques include black-box testing (testing from the specification, without looking at the code), equivalence partitioning, and boundary value analysis.

• Example of functional testing: entering a valid student ID must show that student's record; entering an invalid ID must show a clear error, not crash.

• Performance testing checks how well the system works under load: response time, throughput, and resource use. Types include load testing (expected load), stress testing (beyond expected load, to find the breaking point), and endurance testing (long running load, to find memory leaks).

• Example of performance testing: can the system handle 1,000 users at the same time with a response time under 2 seconds? A system can be functionally correct but still fail if it is too slow under real load.

15. Verification and Validation

Verification and validation (V&V) are the two sides of checking software quality. A classic short form is: verification asks 'Are we building the product right?', validation asks 'Are we building the right product?'

• Verification checks that the product matches its specification at each stage: does the design match the requirements, does the code match the design? Techniques include reviews, inspections, walkthroughs, and static analysis.

• Validation checks that the finished product meets the real needs of users: does it solve the actual problem, in a real environment? Techniques include system testing, acceptance testing, and user testing.

• Verification can be done without running the code (for example, by reviewing a design document). Validation normally requires running the system, often with real users and real data.

• Both are needed: a system can be verified against a wrong specification and still fail validation, because it is not what users actually need.

16. Testing Strategy

A testing strategy is a plan for how testing will be done across the whole project: what will be tested, in what order, by whom, and when testing can stop.

• A common strategy follows the development levels from small to large: unit testing (single functions/classes), integration testing (modules working together), system testing (the

complete system), and acceptance testing (users and customers).

• It also chooses techniques: white-box testing (using knowledge of the code structure), black-box testing (using only the specification), and, where appropriate, automated regression testing so that old faults do not return after a change.

• A good strategy defines test entry and exit criteria. Example of an exit criterion: all critical test cases pass, no open critical defects remain, and test coverage targets are met. Without exit criteria, testing can go on forever or stop too early.

• Testing should start early, not only at the end. Test planning, test case design, and reviews during requirements and design catch faults when they are cheapest to fix.

17. Non-Functional Requirements of Performance

Non-functional requirements describe how well a system must work, rather than what functions it must provide. Performance is one of the most important non-functional requirements.

• Performance requirements are usually stated as measurable targets, not vague wishes. Examples: response time (a search must return results within 2 seconds for 95% of requests), throughput (the system must process 500 transactions per minute), and resource use (memory use must stay below a stated limit).

• Other related performance aspects include scalability (can the system handle growth in users or data?), capacity (what is the maximum load it supports?), and availability under load (does it stay responsive during peak use?).

• Performance requirements must be testable. 'The system should be fast' cannot be tested. 'The home page must load in under 3 seconds on a standard connection' can be tested.

• Meeting performance requirements affects architecture: caching, database indexing, load balancing, and efficient algorithms are design decisions, not afterthoughts added at the end.

18. GUI Design and UI Design Principles

GUI (graphical user interface) design is about how users see and interact with a system on screen. Good UI design makes a system easy to learn, efficient to use, and hard to use wrongly.

• Key principles include: consistency (similar actions look and work the same way throughout the system), visibility (important options and system status are visible), feedback (the system shows what happened after each action), and simplicity (only the information needed for the current task is shown).

• Other principles: user control (users can undo actions and escape from mistakes), error prevention and recovery (clear error messages that explain the problem and how to fix it), and accessibility (the interface can be used by people with different abilities, for example with readable text and keyboard navigation).

• Design process: understand the users and their tasks, sketch and prototype the interface, test it with real users, and revise it. A prototype tested early is far cheaper than fixing a finished interface.

• Example: a form should label every field clearly, show an error next to the field that caused it (not only at the top of the page), and allow the user to go back without losing the data they already entered.

Part 3 — Dependability & Security

19. Security and Maintainability

Dependability means a system can be trusted to deliver its service correctly. Security and maintainability are two key parts of dependability, and they are connected: a system that cannot be maintained securely will become insecure over time.

- Security means protecting the system and its data from unauthorised access, damage, and disclosure. It includes confidentiality (only authorised users can see data), integrity (data is not changed without permission), and availability (authorised users can access the service when they need it).
- Common security practices include authentication (proving who you are), authorisation (checking what you are allowed to do), encryption of sensitive data, input validation to prevent injection attacks, and keeping software updated.
- Maintainability means the system can be changed, fixed, and improved at reasonable cost. It depends on modular design, clear documentation, readable code, and good test coverage.
- The link: if a system is hard to maintain, security patches are slow and risky, so known vulnerabilities stay open longer. Good maintainability therefore supports long-term security.

20. Security Risk Management

Security risk management is the process of identifying, assessing, and reducing risks to a system, so that limited security effort is spent where it matters most.

- Step 1, risk identification: list assets (data, systems, reputation), threats (attackers, malware, insider misuse, natural failures), and vulnerabilities (weak passwords, unpatched software, missing input checks) that could allow a threat to harm an asset.
- Step 2, risk analysis: estimate for each risk its likelihood and its impact. A common tool is a risk matrix that ranks risks as low, medium, or high from these two factors.
- Step 3, risk treatment: choose a response for each important risk. Options are: avoid the risk (remove the feature or data), mitigate it (add a control, such as encryption or access control), transfer it (for example, through insurance or a trusted service), or accept it (when the cost of control is higher than the risk, with monitoring).
- Step 4, monitoring and review: risks change as the system and its attackers change, so the register must be reviewed regularly and after any major change or incident.
- Example: the risk of a database leak from SQL injection is high impact and, without input validation, high likelihood. Mitigation is to use parameterised queries and least-privilege database accounts.

21. Availability and Reliability

Availability and reliability are both about a system working when it is needed, but they measure different things.

- Reliability is the probability that a system will work correctly, without failure, for a stated period under stated conditions. It is often measured by mean time between failures (MTBF) and by failure rate. A reliable system fails rarely.
- Availability is the proportion of time a system is working and accessible when it is needed. It is often written as: Availability = MTBF / (MTBF + MTTR), where MTTR is mean time to repair.

A system can be reliable but still have low availability if, when it does fail, it takes a very long time to repair.

• Example: a system that fails once a year but takes a week to repair has high reliability but lower availability than a system that fails monthly but is repaired in an hour.

• Ways to improve them include: fault avoidance (better design and testing), fault tolerance (redundancy, so one component failure does not stop the system), and fast recovery (monitoring, automatic restart, and good backup and restore procedures).

Part 4 — Software Management

22. Upper CASE, Lower CASE, and Integrated CASE Tools

CASE stands for Computer-Aided Software Engineering. CASE tools are software tools that support the software development process, automate routine work, and help keep documents and code consistent.

- Upper CASE tools support the early stages: requirements analysis, system modelling, and design. Examples: tools for drawing UML diagrams, modelling data, and managing requirements.
- Lower CASE tools support the later stages: coding, testing, and maintenance. Examples: code editors and IDEs, compilers, debuggers, test automation tools, and version control systems.
- Integrated CASE (I-CASE) tools combine upper and lower CASE in one environment, so that a change in a design model can be traced to code, and a change in code can be reflected back in documentation. They usually share a common repository for all project artefacts.
- Benefits: faster routine work, fewer inconsistencies between documents and code, better traceability from requirements to tests, and easier team collaboration. Limits: tools cost money and training time, they do not replace good judgement, and a tool that does not fit the team's process can slow work down.

23. Configuration Management

Configuration management (CM) is the discipline of controlling changes to a software system, so that the team always knows what version of what is in use, and changes do not get lost or applied to the wrong version.

- Key activities: configuration identification (deciding which items are under control, such as requirements, code, tests, and documents, and giving each a version), version control (recording every change, who made it, and why), change control (reviewing and approving changes before they are applied, especially to a released version), and configuration auditing (checking that a build actually contains the approved versions).
- A baseline is an approved version of the system at a point in time, for example a release. Later changes are made as controlled deltas from that baseline, not by editing the release directly without record.
- Modern tools for this include Git and other version control systems, with branches for features and fixes, and pull requests for review before merging.
- Without configuration management, teams face classic problems: two developers overwrite each other's work, a bug fixed in one version reappears in the next, and no one can reproduce the exact version a customer reported a fault in.

24. Software Project Management

Software project management is planning, organising, monitoring, and controlling a software project so that it is delivered on time, within budget, and to the required quality.

- Main activities: project planning (defining scope, breaking work into tasks, estimating effort and time, and scheduling), resource management (assigning people, tools, and budget), risk management (identifying risks and preparing responses), monitoring and control (tracking progress against the plan, and taking corrective action when it slips), and team management

(communication, motivation, and resolving conflicts).

• Estimation is a key skill and a common source of failure. Techniques include expert judgement, analogy with past similar projects, and structured methods such as function point analysis and COCOMO (a cost model based on system size and project factors). Estimates should be revised as more is learned, not treated as fixed promises from day one.

• Project managers use tools such as Gantt charts (tasks over time), PERT charts (task dependencies and critical path), and burn-down charts in agile projects to make progress visible.

• A project can fail even with good code if it is poorly managed: unclear scope, unrealistic schedules, unmanaged change, and poor communication are among the most common causes. Good management does not write the code, but it creates the conditions for the team to write it well.

How to use this file for revision

• Read one question, close the file, and write its definition and 3 key points from memory. Then check what you missed.

• For design and testing questions, always add a short example in an exam — it shows understanding, not just memorisation.

• For definitions that have a formula or a list (for example cyclomatic complexity, or the five OOD activities), learn the formula/list exactly.

• Next step if you want it: I can turn this file into short practice questions with answers, one part at a time.